



HydroSuite-AI: a web-based LLM environment for hydrological code generation and execution for the hydrosuite open-source ecosystem

Vinay Pursnani^{1,2} · Carlos Erazo Ramirez^{3,4} · Yusuf Sermet^{3,5} · Ibrahim Demir^{3,4}

Received: 16 April 2026 / Accepted: 26 August 2026
© The Author(s) 2026

Abstract

Researchers in hydrology face technical barriers from complex models, evolving libraries, and fragmented documentation. We present HydroSuite-AI, an LLM-based assistant that integrates the open-source web hydrologic libraries HydroLang, HydroCompute, and HydroRTC for code generation, factual guidance, and in-browser execution of hydrological workflows. The system uses a Planner-Worker-Synthesizer architecture with retrieval-augmented generation (RAG) over library documentation and validates outputs in a client-side virtualized execution environment. We evaluate HydroSuite-AI through (i) a cross-library case study, (ii) a controlled benchmark measuring the assistant's ability to retrieve and execute library functions across 60 domain-specific tasks under different LLM backends, and (iii) deployment as a code assistant in the WaterSoftHack 2024 workshop. Across the benchmark tasks, HydroSuite-AI achieves 95.0% function retrieval accuracy with 23.1 s mean latency when powered by o3-mini, while the same system reaches 71.7% under 10s with GPT-4.1-nano, showing an accuracy–speed tradeoff across model configurations. The case study demonstrates end-to-end interoperability and exposes limitations in advanced statistical workflows, and the workshop deployment confirms the system's utility in collaborative, time-constrained settings. Documentation-grounded LLM orchestration reduces time to implementation and supports hydrological research and education.

Keywords Hydroinformatics · Multi-agent orchestration · Retrieval-augmented generation · Large language models · Web-based hydrological computing

Introduction

Hydrological research is becoming complex due to sophisticated models required to address current environmental challenges (Krajewski et al. 2021), extensive data analysis, and specialized libraries with limited or evolving documentation

(Zhang et al. 2023; Sit et al. 2021a). Researchers often face training requirements, needing both domain expertise and advanced coding skills to effectively utilize hydrological tools. This dual demand can lead to delays and obstacles in conducting research, as time is spent not on scientific inquiry itself but on mastering necessary computational frameworks.

In parallel, the field of artificial intelligence (AI) has witnessed recent advancements, particularly with the emergence of Large Language Models (LLMs; Sajja et al. 2024). These models have demonstrated capabilities in natural language processing (NLP; Sermet and Demir 2021) and code generation, offering potential solutions to reduce technical barriers faced by hydrologists. By leveraging AI, there is an opportunity to automate workflows, simplify complex tasks, and enhance accessibility to modeling tools without requiring extensive programming expertise (Samuel et al. 2024a).

Given these challenges with hydrological modeling and opportunities with LLM and generative AI (GAIs), we

✉ Vinay Pursnani
vinay-pursnani@uiowa.edu

¹ Department of Informatics, University of Iowa, Iowa City, IA, USA

² IIHR - Hydroscience and Engineering, University of Iowa, Iowa City, IA, USA

³ ByWater Institute, Tulane University, New Orleans, LA, USA

⁴ River-Coastal Science and Engineering, Tulane University, New Orleans, LA, USA

⁵ Pediatrics, School of Medicine, Tulane University, New Orleans, LA, USA

present HydroSuite-AI, an AI-powered assistant designed for the hydrology and environmental science domains. HydroSuite-AI integrates three open-source libraries HydroLang, HydroCompute, and HydroRTC into a unified platform that simplifies hydrological modeling and data analysis. The assistant enables users to generate code snippets, execute computations, and retrieve factual information through natural language queries. The ability to answer factual questions about library usage provides researchers with necessary function details and parameters, reducing time spent searching documentation and enabling quicker, more accurate implementation within analytical workflows.

The system employs a Planner–Worker–Synthesizer multi-agent architecture grounded by retrieval-augmented generation (RAG) over indexed library documentation and validates outputs in a client-side virtualized execution environment. We evaluate the framework through a cross-library interoperability case study and an automated benchmark of 60 tasks across four LLM backends, measuring Function Retrieval Accuracy, hallucination rates, and response latency.

The principal contributions of this work are: (1) a multi-agent orchestration architecture that decomposes natural language queries into library-specific sub-tasks executed by documentation-grounded worker agents; (2) a standalone client-side virtualized execution environment with dynamic dependency injection that validates generated code; (3) a quantitative benchmark comparing four LLM backends across 60 domain-specific tasks spanning three libraries; and (4) a cross-library case study demonstrating end-to-end GIS preprocessing, visualization, and peer-to-peer data sharing.

Background

Potential of LLMs in hydrology

In the past two years, LLMs have proven useful beyond text generation; tasks like sentiment analysis, which once required data collection, labeling, and model training, can now be performed with simple prompts. Models such as OpenAI's GPT series can streamline complex workflows by leveraging learned language patterns and predictive capabilities to extract contextual summaries and correlations from extensive datasets when guided by structured prompts (Bubeck et al. 2023; Gartlehner et al. 2024; Douglas et al., 2023). This shift extends across domains: in healthcare, generative AI supports clinical communication and has produced research text rated comparable to human-written sections (Tai-Seale et al. 2024; Huespe et al. 2023), while in

finance, LLM-driven tools have shown positive impacts on business decision-making (Akour et al. 2024).

Hydrology can benefit from AI, particularly LLMs, by lowering barriers to applying process-based models and analysis workflows (Cooper et al. 2024; Pursnani et al., 2023). LLMs can make technical documentation more accessible through natural language queries and assist with code generation for data access and model implementation. For example, they can help clarify modeling input requirements (e.g., forcings for simulations), GIS formats and inputs, outputs for reanalysis, and acceptance criteria using well-known metrics such as NSE and MSE, enabling more natural prompt-driven interactions with physical modeling tasks. From an educational perspective, LLM-driven conversational agents can also support communication and learning around water quality, improving public engagement in environmental science (Samuel et al. 2024b). Beyond generic conversational assistants, specialized domain-adapted foundation models have recently emerged; notably, WaterGPT demonstrated that supervised fine-tuning on curated hydrological literature and multi-agent tool integration allows LLMs to significantly outperform general-purpose models on domain benchmarks (Ren et al. 2024). More broadly, AI and machine learning have an established trajectory in hydrological applications that contextualizes the emerging role of LLMs.

Examples of automated or assisted workflows include analysis of flood map images and data analysis (Kadiyala et al. 2024; Pursnani et al., 2025), hydrological time series analysis (e.g., streamflow) for seasonal decomposition and trend detection relevant to watershed management (Mosavi et al. 2018; Shahid et al. 2023), and nutrient analysis to assess exceedances of critical water quality thresholds (Herath et al. 2023). Advances in AI-enabled flood early warning systems (FEWS) such as deep learning for improved rainfall-runoff modeling and faster inundation mapping further illustrate the potential for enhancing prediction accuracy and lead times (Li et al. 2023; Baydaroglu et al., 2023). According to Perera et al. (2020), improved satellite sensors, machine learning applied to remote sensing data, and enhanced computational infrastructure have strengthened FEWS capabilities (Sit et al. 2021b). These advances are particularly important in data-scarce regions where infrastructure limitations can restrict deployment and operational effectiveness, potentially affecting forecast reliability (Tegegne et al., 2017). Collectively, these studies demonstrate that AI and LLMs can improve individual hydrological applications, including forecasting, data analysis, and education. They do not consider the broader computational workflow required to conduct hydrological studies, which commonly involves multiple software

libraries with different interfaces and capabilities. As a result, users remain responsible for integrating these tools into a functional workflow. This work addresses that challenge through an LLM-driven framework that assists with workflow construction and execution across hydrological software libraries.

Related work

LLMs are being widely used in software development, driving a significant change in how coding and task execution are approached, with tools like GitHub Copilot and AWS CodeWhisperer leading this transformation (Yeo et al. 2024). LLMs, such as OpenAI's Codex and DeepMind's AlphaCode, have advanced in their capabilities for generating human-like code with assistive comments, enhancing productivity relative to traditional software engineering processes (Wong et al. 2023). For instance, the introduction of Pythia, an end-to-end system that generates ranked lists of methods and API recommendations for developers during coding represents an advancement in workflow deployment. Incorporated within the IntelliCode extension for Visual Studio Code IDE, Pythia leverages large-scale deep learning models trained on code contexts derived from abstract syntax trees. The ability of these models to understand and generate code from natural language prompts enables natural language interaction between developers and coding tools, making programming more accessible, especially for those with limited experience. However, research also indicates that while LLMs can produce high-quality outputs, generated code may contain errors or security vulnerabilities, particularly when training data includes flawed or unsafe code (Al-Kaswan and Izadi 2023; Pan 2024; Improta 2023).

To address these shortcomings, RAG has proven effective. With RAG, external data is retrieved and injected into the prompt context window, enabling integration of disparate sources and complete separation of data from the model (Peng et al. 2023; Lewis et al. 2020; Gao et al. 2023). The self-attention mechanism introduced in the Transformer architecture allows LLMs to weigh the importance of different tokens within the context window, capturing relationships and dependencies necessary for deep language understanding (Vaswani et al. 2017). Larger context windows up to 128 K tokens enhance capabilities and enable previously unattainable tasks (Song 2023). Leveraging documentation for programming libraries within this setup supports efficient code generation and productivity. OpenAI's Codex and ChatGPT were trained on extensive datasets that include both natural language and code, allowing them to generate code based on user

specifications and context (Le 2024; Zhong 2024). This is particularly beneficial when developers need assistance adhering to specific libraries or frameworks, as LLMs can interpret and incorporate documentation nuances into generated code (Zhong 2024; Liu 2024).

Utilizing factual and code-like responses from AI remains an active research area due to nuanced failure modes. Liu et al. (2024) provide a comprehensive analysis of hallucinations in LLM-powered code generation, defining them as cases, where the generated code deviates from user intent, factual knowledge, or code context. The work proposes a taxonomy with five primary categories and 19 specific types, highlighting the challenges of recognizing and mitigating hallucinations even with advanced prompting techniques.

Specifically, the five categories are Intent Conflicting (code that contradicts the user's stated request), Context Inconsistency and Context Repetition (code that is internally inconsistent or unnecessarily repetitive relative to the surrounding context), Dead Code (unreachable or non-functional code fragments), and Knowledge Conflicting (code that misuses or misstates factual API or library-specific knowledge, such as function names or parameters). For the present evaluation, we grouped Context Inconsistency, Context Repetition, and Dead Code under a single Context Deviation category, given their shared relevance to code-context faithfulness, while retaining Intent Conflicting and Knowledge Conflicting as distinct categories most relevant to documentation-grounded, multi-library code generation.

Methodology

Scope and purpose

HydroSuite-AI is designed to meet the operational needs of hydrologists and environmental researchers by bridging the gap between domain expertise and web-based computational implementation. Rather than functioning as a standard chatbot, HydroSuite-AI integrates three open-source web libraries HydroLang, HydroCompute, and HydroRTC into a Multi-Agent Orchestration Framework. This architecture enhances research productivity by decomposing complex natural language queries into executable, context-aware workflows.

This paper examines the framework's Planner-Worker-Synthesizer paradigm, its virtualized execution capabilities, and the quantitative benefits observed through case studies. The development was guided by the following research questions:

RQ1 Can a multi-agent orchestration system generate code snippets that integrate with existing user-specific workflows?

RQ2 Can specialized worker agents accurately answer factual questions regarding complex and evolving software library documentation?

RQ3 Can a synthesizer agent provide contextually relevant and interoperable responses to multifaceted hydrological queries?

RQ4 Does the virtualized execution environment improve interoperability across various data formats and tools?

Overview of hydrosuite libraries

HydroSuite is a collection of web-based libraries designed for hydrology and environmental domains, leveraging open-source, distributed architectures accessible through web browsers. Libraries promote hydrologic data open access and analysis by addressing data management, scientific computing, communication, and community portals. The objective is to enable end-to-end workflows through efficient data integration by acquisition and dissemination, fostering collaboration, knowledge sharing, and innovation.

Key libraries and applications include HydroLang, HydroCompute, and HydroRTC. HydroLang is a modular library offering tools for data fetching and dissemination, statistical and hydrologic analysis, geospatial processing for gridded sources, modeling, and spatiotemporal analysis. It provides visualization capabilities (maps, charts, tables) and connects global data sources (Erazo Ramirez et al. 2022). Extensions such as HydroLang-ML (Erazo Ramirez et al. 2023) and HydroLang-BMI (Ewing et al. 2024) further enhance data open access and broaden usability. HydroCompute is a high-performance computing library utilizing modern web technologies to support multithreading on the client side, optimizing resource use on user devices. It includes web support for C, C++, WebGPU, AssemblyScript, and JavaScript, along with emerging WebAssembly engines running Python and R in the browser. The library leverages indexed acyclic graphs and algorithmic optimizations to achieve client-side, low-level computing (Erazo Ramirez et al. 2024a). HydroRTC facilitates large-scale data sharing and transfer between peers and servers through decentralized WebRTC and WebSocket technologies. It enables efficient transmission, streaming, task allocation for complex data, and distributed computing, with task-specific resource requests. It supports common hydrologic formats such as NetCDF, GRIB, and HDF5 (Erazo Ramirez et al. 2024b). All libraries referenced in this manuscript include documentation, case studies, and publicly

accessible training materials. Interoperability is maintained through open-source implementation and adoption of common communication standards such as the Basic Modeling Interface, event buses, and message passing interfaces.

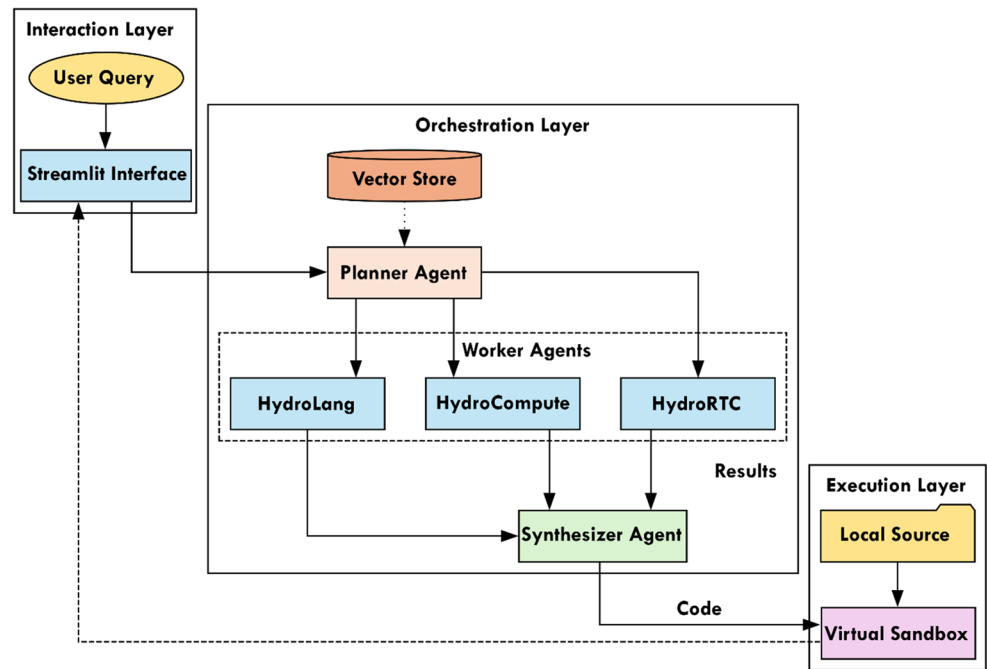
Multi-agent orchestration architecture

HydroSuite-AI is an integrated AI-powered assistant that provides automated assistance for handling factual queries and executing code, using the HydroSuite libraries and examples. The system is organized into four main modules: External Libraries, HydroSuite UI, Data Sources, and HydroSuite Core. These four modules map onto the system's functional architecture as follows: the HydroSuite UI module corresponds to the Interaction Layer in Fig. 1 (the Streamlit interface); External Libraries corresponds to the Worker Agent instances grounded in scoped libraries' documentation; Data Sources corresponds to the Local Source component feeding the Execution Layer; and HydroSuite Core encompasses the remaining backend components, the Planner Agent, Vector Store, Synthesizer Agent, and Virtual Sandbox, that together implement the three functional parts described below. When a user submits a request, the Streamlit UI captures the query and forwards it to the Flask-managed backend. The backend determines whether the query seeks factual information or requires code execution. Regardless of type, the query is transformed into vector embeddings and compared against indexed documentation in the vector store, retrieving the 20 most relevant chunks to provide context. An architectural overview is shown in Fig. 1.

HydroSuite-AI advances beyond standard linear chatbots by implementing a Planner–Worker–Synthesizer architecture (Fig. 1). This hierarchical framework decouples task decomposition from execution and enables coordination across distinct libraries for multifaceted hydrological workflows. The architecture comprises three primary parts:

Orchestration layer (planner & workers) The system first classifies intent. For complex computational requests, a Planner Agent analyzes the prompt and generates a strict JSON Virtual Execution Environment (VEE) plan that decomposes the user's intent into library-specific sub-tasks (e.g., retrieve a specific data source vs. perform a geospatial operation). These sub-tasks are dispatched asynchronously to specialized Worker Agents using a parallel thread pool executor. Each Worker Agent is an instance of the LLM configured and grounded via RAG on the documentation for its target library, which is to aid with mixing of API syntaxes. For factual queries, the system returns a clear textual response. For code execution, relevant code snippets are assembled, converted into executable files, and executed within the virtual sandbox; results are then presented to the user alongside interactive VEEs for further exploration and refinement.

Fig. 1 Architecture overview of the HydroSuite-AI multi-agent orchestration framework



Knowledge retrieval and vectorization (RAG) To ground Worker Agents in accurate technical data, the system employs a Retrieval-Augmented Generation pipeline. Documentation and usage examples for all three libraries are preprocessed into semantic chunks to optimize retrieval accuracy. Configuration details: embeddings at 256 dimensions; chunk size of 1200 tokens with 150-token overlap; cosine-similarity retrieval of the top 20 chunks. These values were selected following informal preliminary testing across candidate configurations, in which chunk size, overlap, and retrieval depth were varied and qualitatively evaluated based on whether the retrieved chunks contained sufficient information to correctly answer a set of representative factual and code-generation queries. This testing was exploratory; the 256-dimensional embedding size was adopted primarily for computational efficiency, a tradeoff we revisit in the Limitations section. During inference, retrieved chunks are injected into the model context (e.g., o3-mini), ensuring access to exact function signatures and parameter requirements for the specific task. The knowledge ingestion and RAG pipeline is summarized in Fig. 2.

Contextual synthesis and scaffolding Outputs from the parallel Worker Agents are aggregated by the Synthesizer (Master Orchestrator) Agent. Rather than concatenating text, the Synthesizer acts as a Lead Architect, operating under a strict assembly protocol that scaffolds the final application structure (Table 1).

This scaffolding performs three critical functions. Dependency normalization is achieved by enforcing specific

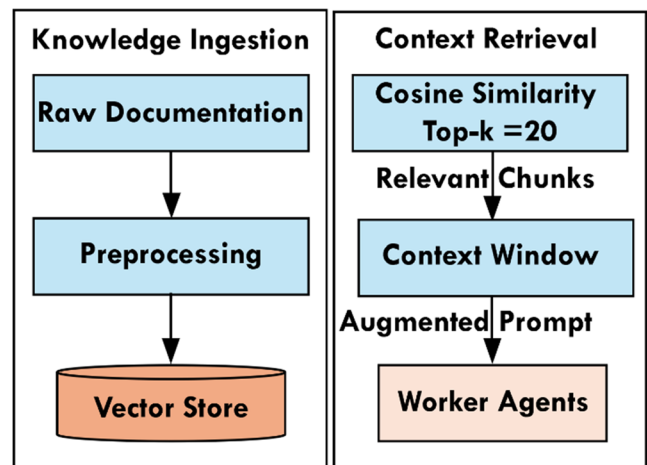


Fig. 2 Knowledge ingestion and RAG pipeline

relative file paths matching the virtualized execution environment, ensuring the browser VEE can locate the library dependencies offline. The hallucination safety checks are done through the orchestrator that scans the reports for DOCS_MISSING flags. If a feature is unsupported by the documentation, the scaffolding logic prevents the generation of non-functional code and instead injects a user warning. Moreover, the scaffolding creates cohesion enforcement by merging isolated logic snippets (e.g., a chart configuration from HydroRTC and a data fetch from HydroLang) into a single, unified HTML/JavaScript boilerplate.

In Fig. 1, the Orchestration Layer corresponds to the Planner Agent and its downstream Worker Agents; Knowledge Retrieval and Vectorization corresponds to

Table 1 System prompts and constraints for the master orchestrator (synthesizer) agent

Instruction component	Directives and constraints
Role Definition	“You are the Lead Architect & Assembler. Manage interaction between HydroLang, HydroCompute, and HydroRTC specialist agents.”
Phase 1: Planning	“Analyze user requests and explicitly state which specialists are needed. Create a technical step-by-step plan for execution.”
Phase 2: Assembly	“Receive technical research from specialists. Merge findings into a single, cohesive, functional one-page HTML/JavaScript application.”
Path Scaffolding	“Search documentation for correct tags. Use these relative paths matching the local virtual file system: ./libs/hydrolang/hydrolang.js, ./libs/hydrocompute/src/hydrocompute.js, ./libs/hydrortc/hydrortcserver.js.”
Safety Protocol	“If a specialist reports ‘DOCS_MISSING’, do NOT hallucinate the code. Add a visible warning note at the top of the generated HTML indicating the missing feature.”
Output Format	“Always wrap the final code in html’ and javascript’ blocks for parsing by the execution environment.”

the Vector Store that supplies retrieved context to the Planner Agent; and Contextual Synthesis and Scaffolding corresponds to the Synthesizer Agent together with the Virtual Sandbox in the Execution Layer, where assembled code is executed.

Virtualized execution environment

To validate the synthesized code, HydroSuite-AI implements a client-side Virtualized Execution Environment (VEE). Upon execution, the system recursively scans the local source directories of the HydroSuite libraries on the host server. These files are serialized and injected directly into the browser-based Integrated Development Environment as a Virtual File System. This approach validates the structural integrity and ability to function in an offline-capable, self-contained directory structure.

Evaluation protocols

This section describes the protocols used to evaluate HydroSuite-AI across three complementary dimensions: a qualitative expert assessment of responses to characterize failure modes and response quality, a cross-library interoperability case study, and an automated quantitative benchmark serving as primary performance evaluation. All results produced under these protocols are reported in Sect. 4.

Qualitative expert assessment protocol

For the qualitative assessment, we curated a set of prompts spanning factual questions and code-generation tasks across the specified libraries. Each system response was rated on a 1–5 scale, where 1 indicated an unhelpful or incorrect response and 5 indicated a correct and useful answer that addressed the user’s request. To characterize failure modes, we applied the hallucination taxonomy described by Liu et al. (2024), which classifies deviations from user intent or documentation-grounded facts into three main groups: Intent Conflicting, Context Deviation (including subtypes such as Inconsistency, Repetition, and Dead Code), and Knowledge Conflicting (including API Knowledge and Identifier Knowledge). We also assigned a complexity class to each prompt to reflect task breadth: A (low complexity, typically direct factual questions grounded in documentation), B (medium complexity, generally single-library tasks requiring examples or basic integration), and C (high complexity, often involving multi-step workflows, multi-library orchestration, or translation of examples to real-data usage). All prompts were executed under the same RAG configuration described in Sect. 3.3 to ensure consistent access to documentation and examples during generation. The complete list of prompts used in this assessment is provided in the Supplementary Materials. Results, including expert scores and categorical error labels, are presented in Tables 3 and 4 and discussed in Sect. 4.2.

Interoperability case study protocol

To evaluate cross-library orchestration (RQ4), we tasked HydroSuite-AI with generating a Collaborative Peer-to-Peer Geospatial Analysis Tool that integrates HydroLang, HydroCompute, and HydroRTC within a single synthesized one-page HTML/JavaScript application. The target workflow required (i) retrieving and processing gridded datasets on the client side, including reprojection of high-resolution rasters to a web-compatible CRS, (ii) rendering the resulting layers on an interactive web map, and (iii) serializing and transferring processed raster buffers to a peer over a decentralized channel for collaborative visualization and analysis. The generated application was executed as described in Sect. 3.4 to confirm offline-capable bundling of dependencies, and verify end-to-end functionality. Success was defined as correct execution of the full dataflow fetch/process in HydroCompute (including use of HydroLang modules as required), render in HydroLang, and peer-to-peer transmission in HydroRTC together with consistent variable and object handling across library boundaries. The

exact prompt used to initiate this case study is provided in the Supplementary Materials. Results and synthesized interface are presented in Sect. 4.3 with Fig. 7.

Automated benchmarking protocol for LLM backends

To evaluate the reliability and efficiency of the HydroSuite-AI orchestration architecture, we implemented a high-throughput automated benchmarking pipeline to quantify performance across multiple LLM backends. The evaluation used a curated dataset of domain-specific tasks designed to test the system's capabilities across three key dimensions: complex data retrieval via HydroLang, computational orchestration via HydroCompute, and peer-to-peer collaboration via HydroRTC. The prompt was verified by the lead developers of the HydroSuite libraries to ensure every requested task corresponds to a documented feature, effectively serving as a ground-truth answer sheet with expected primary functions for each query. While the prompts avoid naming target functions directly, they are phrased using domain-specific hydrological computational terminology (e.g., “Mann-Kendall trend test,” “D8 algorithm,” “Hargreaves method”) that assumes intended user has some understanding of hydrological concepts and computational hydrology. Therefore, this platform benchmarking does not fully account for how the system would handle non-expert user queries. The benchmarking process followed a two-stage pipeline: (i) *Generation Phase*: A parallel execution engine using Python's concurrent features issued asynchronous requests to the HydroSuite-AI backend across four GPT Models: GPT-4o-mini, o3-mini (with medium reasoning effort), GPT-4.1-nano, GPT-4.1-mini. This phase captured the Mean Response Latency (L_{avg}) defined as the wall-clock time from request submission to the receipt of the synthesized code. (ii) *Automated Verification Phase*: To assess RAG performance and mitigate hallucinations, we utilized an o3-mini model to evaluate the generated code against a dataset of 60 domain-specific tasks. The model was instructed to assess outputs based on functional equivalence, marking code as valid if it achieved the intended result using correct library aliases or wrappers, while rejecting outputs with method mismatches (e.g., mixing data source request or factually using incorrect methods for analyzing datasets). We defined Function Retrieval Accuracy (FRA) as the percentage of responses correctly identifying the target library functions compared to the ground truth. Furthermore, the Function Hallucination Rate (FHR) represents the percentage of responses referencing non-existent or incorrect function identifiers. Table 2 provides a representative sample of the evaluation dataset, illustrating the comparison between user queries and expected library functions. The complete verification dataset is available in the Supplementary Materials.

Table 2 Sample of the verification dataset comparing user queries against expected library functions (subset of $N = 60$).

Task ID	Library	User query	Expected function
1	HydroCompute	Calculate a simple moving average with a window size of 10 for the dataset 'stream_level'.	<i>simple-MovingAverage.js</i>
8	HydroCompute	Use WebGPU to multiply the large matrices identified as 'large_mat_x' and 'large_mat_y'.	<i>matrix-Multiply_gpu</i>
21	HydroLang	Fetch real-time stream-flow data for USGS site 05454500 from the last 7 days using a JSON format.	<i>data.retrieve</i>
40	HydroLang	Display a GeoTIFF elevation raster on my map using a terrain color scheme with 70% opacity.	<i>map.addGeo-Raster-Layer</i>
47	HydroRTC	Establish a direct peer-to-peer connection with a remote user named 'Peer_B'.	<i>connect-Peer</i>

Results and discussion

Features and functionality

The system supports hydrological research by integrating the libraries HydroLang, HydroRTC, and HydroCompute. Its interface provides clear navigation across modules tailored to specific tasks in hydrological analysis and data processing. As shown in Fig. 3, users can access these modules directly, each targeting a distinct component of the workflow.

One supported task type is factual question answering, which enables users to query hydrological concepts, environmental science topics, and library-specific functionality. The system processes complex queries and retrieves relevant information from its indexed knowledge base. For example, as shown in Fig. 3, a query about HydroCompute returns a structured explanation of its features and applications based on its documentation.

Beyond general explanations, this feature supports workflow integration by returning implementation-ready details. As shown in Fig. 4, when a user asks how to generate synthetic rainfall data with HydroLang, HydroSuite-AI identifies the relevant function (*stochasticRainfallGeneration*), lists required inputs (e.g., *observedRainfall* and optional *distributionType*), and provides example code. With inputs and usage defined, users can integrate the function directly

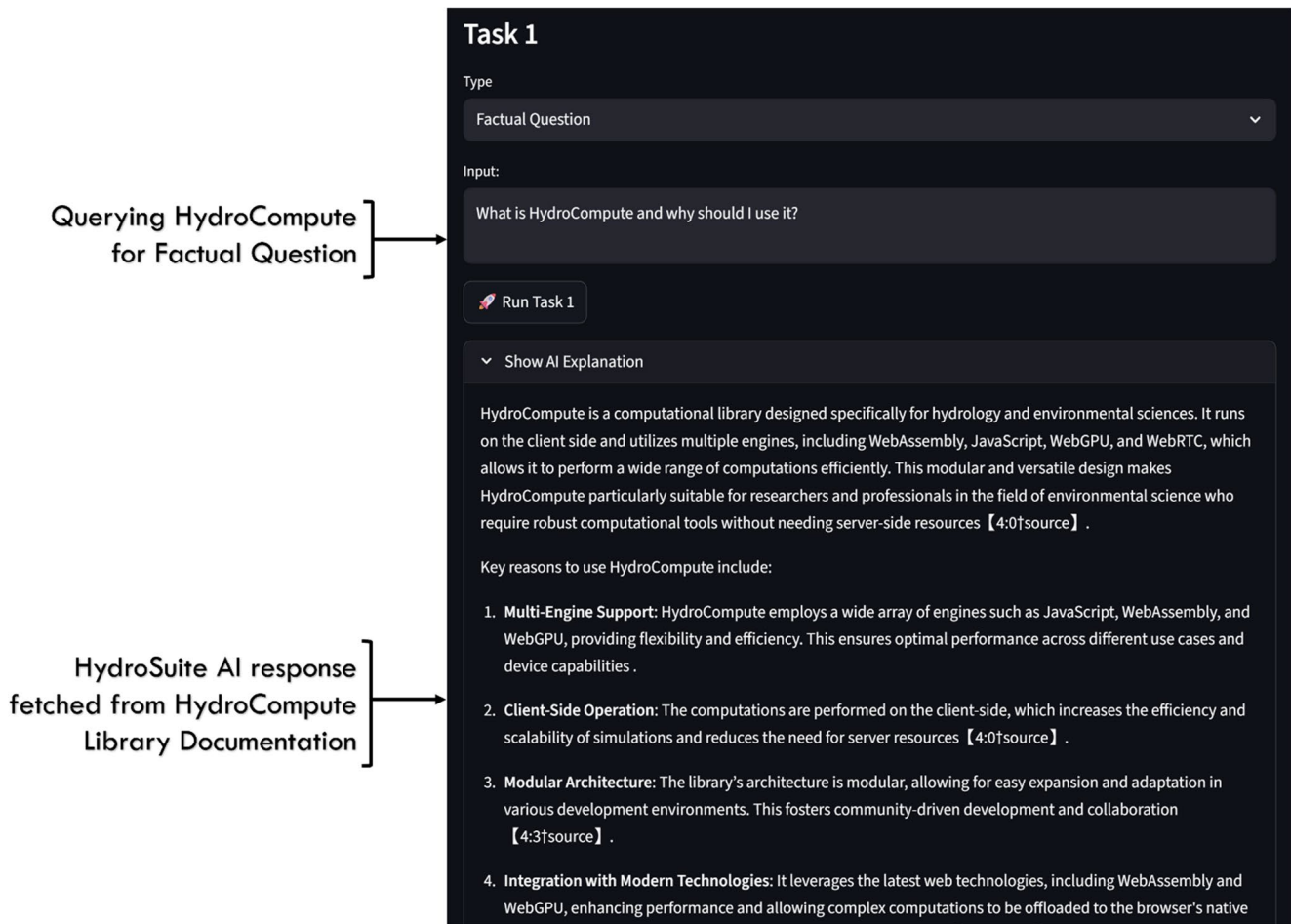


Fig. 3 Task execution interface showing a response to a general factual query about HydroCompute

into their workflow without resolving low-level implementation details. For example, when generating a flood hydrograph from synthetic rainfall, the user focuses on applying the method rather than constructing it.

This interaction illustrates the system's role in bridging documentation and execution. By returning function names, parameter specifications, and code examples in response to targeted queries, HydroSuite-AI reduces time spent searching documentation and accelerates integration into analysis workflows. However, the generated outputs are not always fully reliable. As shown in the qualitative assessment below, parameter structures in code examples may require user verification.

The Code Execution option generates and runs code from natural language queries. For example, the request “fetch and plot USGS daily values for site ID 10010000 in 2022” produces the corresponding data retrieval and plotting code. As shown in Fig. 5, HydroSuite-AI returns the code and executes it in a client-side VEE. The VEE allows users to run, edit, and re-run generated code within an isolated environment. Outputs update immediately, supporting

inspection and modification during execution. Figure 6 shows the editable interface with user queries and results. The interface also integrates documentation, examples, and user feedback on generated outputs, supporting iterative refinement of both code and system responses.

Qualitative expert assessment

As a characterization of response quality and failure modes, four prompts of varying complexity were evaluated for each HydroLang (Table 3) and HydroCompute (Table 4) with the following categorization on error labels: [1] Intent Conflicting; [2] Context Deviation; [2][1] Context Inconsistency; [2][2] Context Repetition; [2][3] Dead Code; [3] Knowledge Conflicting; [3][1] API Knowledge; [3][2] Identifier Knowledge. Throughout these evaluations the system tended to hallucinate when more examples were needed in the training datasets. In terms of context, the main issue with code generation was the system's difficulty in following patterns, especially when several library implementations required clear decisions to make sense of their usage.

User query requesting function and parameters for workflow task.

Input:

How can I generate random numbers that mimic the statistical distribution of rainfall, and what inputs do I need for this process?

[Run Task 1](#)

HydroSuite AI response providing implementation details from HydroLang documentation

▼ Show AI Explanation

To generate random numbers that mimic the statistical distribution of rainfall using HydroLang, you should use the method `hydroLang.data.stochasticRainfallGeneration`. Here is how you can proceed:

1. Load HydroLang via CDN in your HTML:

```
<script src="https://unpkg.com/hydroLang"></script>
```

2. Required Inputs:
 - `observedData`: An array of observed rainfall data (historical rainfall observations).
 - `distribution`: The statistical distribution type (e.g., "normal", "gamma", etc.).
3. Example usage:

```
const observedRainfall = [23, 15, 12, 8, 0, 0, 35, 42, 10, 5]; // Example observed data
const distributionType = "gamma"; // Or "normal", depending on target distribution

const simulatedRainfall = hydroLang.data.stochasticRainfallGeneration({
  observedData: observedRainfall,
  distribution: distributionType
});

// Output variable for HydroCompute:
let output_simulatedRainfall = simulatedRainfall;
```

Identified Function Name, Required Parameters, Provided and Example Usage Code for Integration

Summary of required inputs:

- `observedData` (Array): Historical rainfall values.
- `distribution` (String): Type of statistical distribution to mimic (e.g., "normal", "gamma", etc.).

Fig. 4 Example interaction showing HydroSuite-AI answering a factual query about generating synthetic rainfall data in HydroLang, including the function, required inputs, and example usage

However, the system was able to retrieve correct information and comprehend information from the trained materials for each library, utilizing the pre-context from the GPT training system to incorporate other important features not explicitly mentioned in the libraries. This provides a structured basis for failure-mode characterization ahead of the primary automated benchmark reported in Sect. 4.4. The full prompts are provided in the Supplementary Materials.

The LLM outputs contain structural errors that affect reliability. In Fig. 4, HydroSuite-AI correctly identifies the `stochasticRainfallGeneration` function but misrepresents its input structure. The response lists `observedRainfall` as a required parameter inside a `params` object, whereas the actual function expects the input under a top-level `data` key, with `params` used only for configuration (e.g., `{ data: observedRainfallArray, params: { distributionType: '...' } }`). This mismatch reflects a misinterpretation of the library's API rather than a missing capability. As a result, generated

code requires verification against source documentation or execution before use in workflows. These errors illustrate limitations in documentation-grounded LLMs and highlight the need for better-structured examples and training data for LLM-based assistants (Liu et al. 2024).

Interoperability assessment

To validate HydroSuite-AI's ability to orchestrate cross-library interactions (RQ4), we tasked it with generating a Collaborative Peer-to-Peer Geospatial Analysis Tool. This required the simultaneous integration of HydroLang, HydroCompute, and HydroRTC to fetch, process, visualize, and share hydrological raster data. The exact prompt used for this case study is provided in the Supplementary Materials.

The orchestration engine produced a multi-step workflow (Fig. 7). For data retrieval and processing with

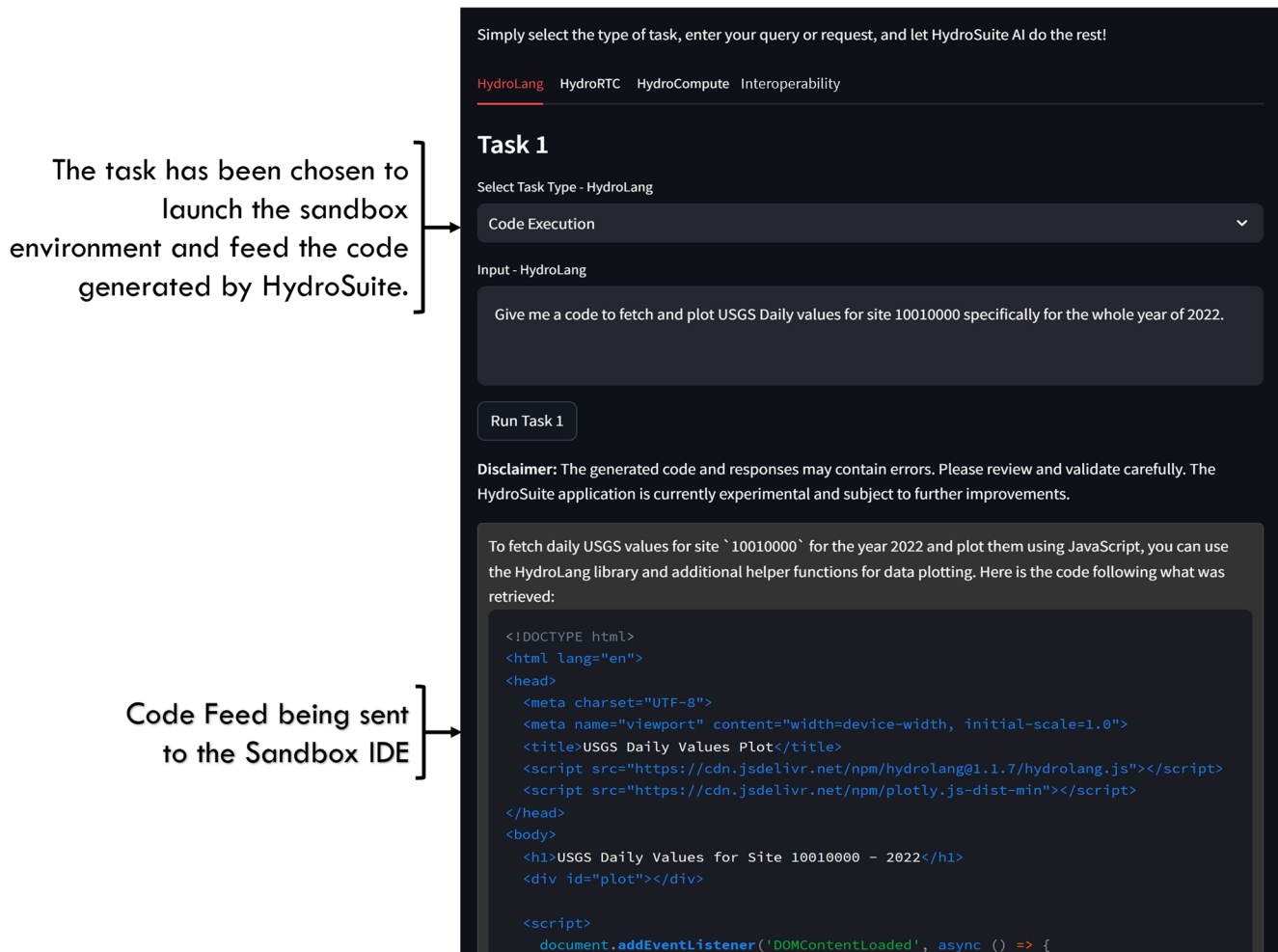


Fig. 5 Code generation and interpretation interface for HydroLang with sample output

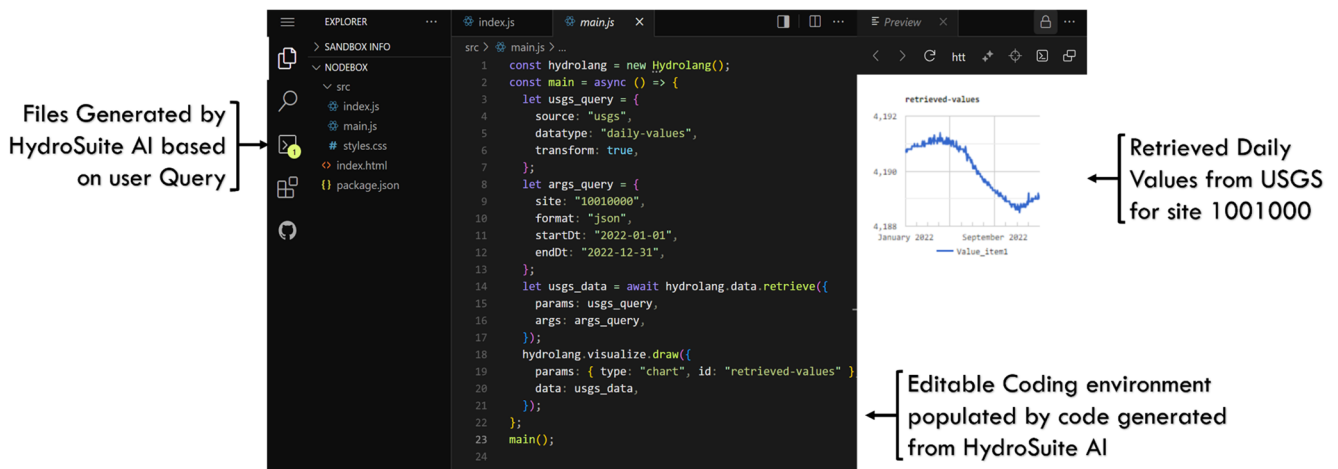


Fig. 6 Code with editable coding environment and user query and results

HydroCompute, HydroSuite-AI generated a data-handlers. js module that uses HydroLang through HydroCompute's retrieve function to fetch high-resolution elevation data from

3DEP (USGS, 2023) and precipitation grids from CHIRPS (Funk et al. 2015). HydroLang's geoprocessor module, leveraging GDAL, processed the rasters, and a reprojection

Table 3 HydroLang evaluation results with question text, score and error

Question Text	Type & Complexity	Expert Score	Categorical Error	Description and Reasoning	Possible Corrections
What is HydroLang and how is it used?	Factual & A	5	None	Correctly prompts the input based on what's asked.	None required.
What are the different data sources available through HydroLang?	Factual & B	3	[3] [3] [1]	The response fails to capture several data sources available within the library.	Retrain the model with comprehensive data source examples.
How do I put a map in HydroLang and connect to a USGS station	Code & C	3	[3] [3] [2]	The bot generates map code correctly but struggles with connecting to external data sources linked to other modules.	Provide more examples on linking data sources and modules.
Write code for retrieving data using HydroLang retrieve function using the WaterOneFlow data source and the GetSitesByBoxObject datatype for the USGS in the box east: -111.38, west: -112.46, north: 41.07, south: 40.19 for the USGS Unit Values	Code & C	2	[2] [2] [3] [3] [1] [3] [2]	Correctly uses functions for data retrieval but shows issues in parameter descriptions.	Train the system with new examples and updated library releases.

kernel transformed their coordinate reference system into a web-compatible CRS using client-side WebAssembly. For visualization with HydroLang, the application initialized a Leaflet-based map (*hydro.map.renderMap*) and correctly parsed the binary raster data from HydroCompute, rendering it as interactive, georeferenced layers.

To enable decentralized data sharing, the system used HydroRTC to establish a collaborative chat interface. Users could connect via P2P, and distributed computing handled tasks such as reprojection across connected participants. The generated code (*hydroRtcClient.peerConn.send*) serialized the processed raster buffers and transmitted them directly to peers, allowing User B to visualize data fetched and processed by User A without a server intermediary. The chat interface can be further customized by user prompts, and the application's layout and styling leveraged both AI examples and general knowledge from the AI's knowledge base.

This case study demonstrates that the synthesizer agent successfully maintained variable consistency across libraries. Figure 7 shows a sample interface where User A fetches and reprojects 3DEP elevation data, pairs with User B via HydroRTC, and User B receives and visualizes the terrain analysis locally.

Comparative performance of LLM backends

The benchmark results, summarized in Table 5, reveal performance profiles for different hydrological workflows, highlighting trade-offs between model size, inference speed, and adherence to retrieved documentation. By isolating the FRA, we observed how different model architectures prioritize retrieved context versus pre-trained knowledge.

The o3-mini model emerged as the most reliable backend for HydroSuite-AI, achieving a Function Retrieval Accuracy (FRA) of 95.00%. This high accuracy indicates a stronger adherence to retrieved documentation and selecting correct function identifiers, making it the most suitable model for workflows where high precision is required. However, this precision comes with the highest L_{avg} of 23.14 s. GPT-4o-mini followed with an FRA of 86.67% and a L_{avg} of 20.96 s, maintaining high reliability in adhering to retrieved documentation. In terms of efficiency, the GPT-4.1-nano model demonstrated the highest performance speed, with a L_{avg} of 9.98 s which is less than half that of the top-performing model while maintaining an FRA of 71.67%. This performance profile presents GPT-4.1-nano as the fastest model tested and may be a suitable choice for interactive, real-time applications where speed is prioritized, and minor errors can be iteratively corrected by the user.

Table 4 HydroCompute evaluation results with question text, score and error

Question Text	Type & Complexity	Expert Score	Categorical Error	Description and Reasoning	Possible Corrections
What is HydroCompute and how do I use it?	Factual & A	5	None	Correctly prompts the input based on what's asked	None required.
How can I create a new set of functions in C so I can use them within HydroCompute?	Factual & C	2	[2] [2]/[1] [3] [3]/[1]	The first part of the question was correctly answered. However, the second part in porting code into the HydroCompute uses a separate way of integrating code.	Provide more detailed examples of code integration to avoid inconsistencies and improve API knowledge.
Given the existing examples for function running within the HydroCompute WASM module, how can I create a MonteCarlo-ARIMA model with real data using the library?	Code & C	4	None	Correctly assesses the process for generating code using the existing functions from the available library within HydroCompute. Generic solutions are given instead of direct examples. However, the response highlights the structure necessary for HydroCompute execution which provides direct guidance.	Improvement with the use of more specific examples during training could enhance the answer.
Write code that shows how I can retrieve data using HydroLang from USGS at a streamflow station obtaining data from 100 years of daily hourly streamflow creating simple moving average from HydroCompute JS functions and display the results in a graph.	Code & C	3	[2] [2]/[1] [3] [3]/[1]	The code generated was correct in the call for HydroLang and partially correct for HydroCompute, particularly in using existing functionalities. Code would work but lacks extensibility.	More specific prompts and examples on particular uses for the modules would ensure the generation of semantically correct code.

User testing and feedback

In July–August 2024, HydroSuite-AI and its associated libraries were showcased at WaterSoftHack 2024 (WaterSoftHack, 2024), a two-week online hackathon supported by a 3-year NSF project. The event aimed to provide students, practitioners, researchers, and educators in hydrology and environmental sciences with tools for data analytics, cyberinfrastructure, and machine learning. Participants came from hydrology departments and universities worldwide, coordinated by Clemson University, The University of Iowa, and CUAHSI.

During the event, participants received training in web development using HydroSuite libraries. HydroSuite-AI served as an interactive assistant, helping users answer questions about the libraries, generate code snippets, and clarify workflow integration. Engagement began with approximately 120 participants and gradually narrowed to 10 fellows, who were selected to develop projects using the libraries.

Throughout the hackathon, HydroSuite-AI guided participants in creating web-based applications, explained the connections between library components, and provided step-by-step support for coding tasks. Qualitative feedback highlighted that participants found it easier to access relevant documentation and apply library features in their projects. The AI was particularly effective at answering factual questions and assisting with code generation, supporting hands-on learning and deeper understanding of web-based hydrological workflows (Hicks et al. 2024). The hackathon results point to a specific bottleneck. Participants spent time locating the correct function signature, argument order, and data format across three actively developed libraries before modelling work could begin. The FRA and FHR results in Table 5 reflect the same pattern, and HydroSuite-AI is best understood as addressing this lookup step instead of direct modelling or interpretation that follows it.

Under documentation-grounded retrieval, the assistant resolved this lookup step correctly between 71.67% and 95.00% of the time, depending on backend (Table 5). The

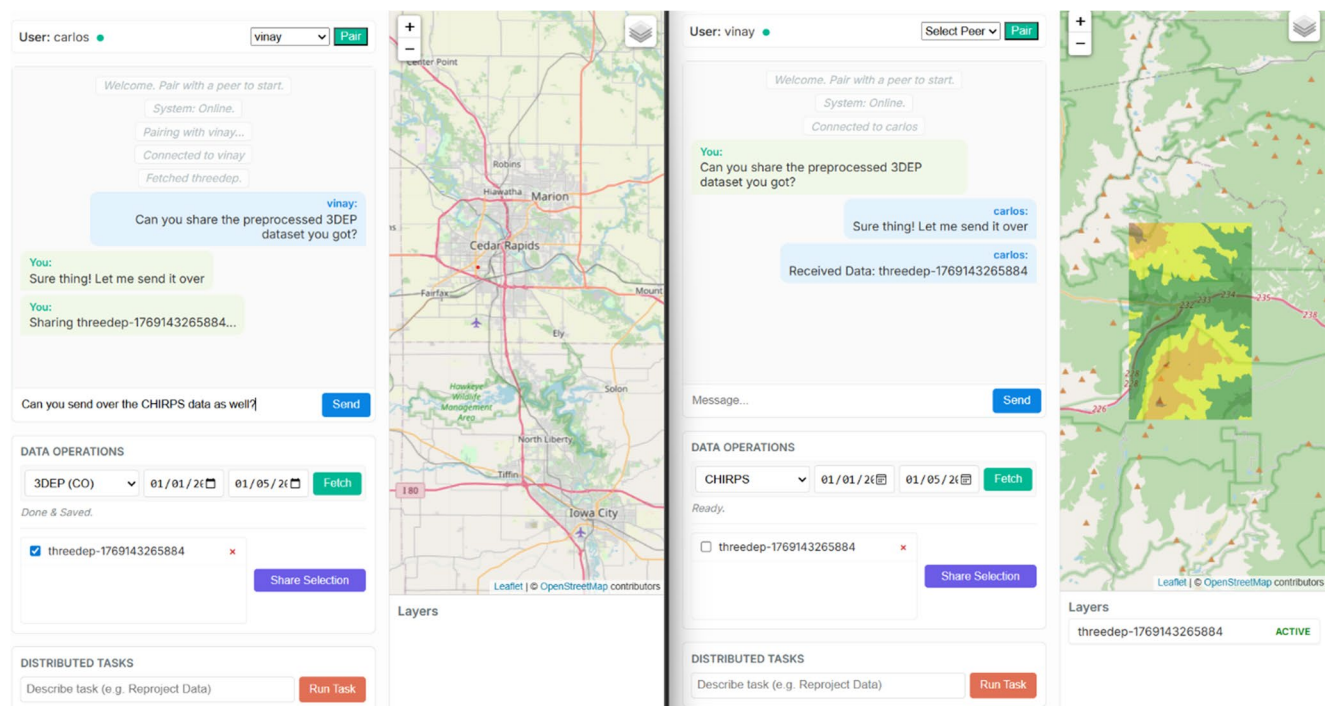


Fig. 7 Sample interface for a collaborative application generated by HydroSuite-AI. (Left) User A fetches and reprojects 3DEP elevation data and User A pairs with User B via HydroRTC. (Right) User B receives the binary data via P2P transfer and visualizes the terrain analysis locally

Table 5 Comparative performance benchmarks of the GPT models evaluated across the app

Model	FRA (%)	FHR (%)	L_{avg} (s)
o3-mini	95.00%	5.00%	23.14
GPT-4o-mini	86.67%	13.33%	20.96
GPT-4.1-mini	73.33%	26.67%	15.67
GPT-4.1-nano	71.67%	28.33%	9.98

interoperability case study (Fig. 7) shows the same mechanism holding across a full workflow spanning three libraries rather than a single isolated function call.

This is a bounded form research; for hydrologists who already know what analysis they want to run but not the specific library call, the tool shortens the path from intent to a first working draft of code, which the user still edits, tests, and validates as with any AI-generated code (Xu et al. 2022). It does not remove the need for domain judgment or code review as failure modes may still occur including hallucinated function usage and incomplete parameters on multi-library tasks. This means generated code should be treated as an initial working draft moving towards a finished deliverable (MacNeil et al. 2023). Accordingly, the benefit is largest for early-career researchers and students with partial coding backgrounds who need to move quickly from question to functioning code, aligning with broader computing education findings where AI code generators act as effective virtual tutors for novices but demand active code review to mitigate subtle hallucinations (Cambaz and Zhang

2024) and smallest for complex, multi-step workflows, consistent with where the benchmark and case study both show reliability declining.

The function-lookup benefit demonstrated here does not by itself establish research acceleration outside the cases studied, however, it demonstrates how multilibrary execution can be done through an agentic scope. A controlled comparison of assisted and unassisted development time would be needed to make that case and is proposed as future work.

Discussions

The evaluation of HydroSuite-AI provides insights into the key research questions guiding this study. In response to RQ1, the results from code generation and execution tasks indicate that HydroSuite-AI is capable of producing functional code snippets that integrate effectively with user workflows, particularly for straightforward tasks. For example, the system successfully generated JavaScript code which executed correctly within the virtual execution environment (Fig. 6). However, when tackling more complex tasks, such as integrating multiple libraries or combining different data sources, the assistant occasionally produced incomplete parameter descriptions or hallucinated function usage. This suggests that while HydroSuite-AI can accelerate simple workflow implementation, additional refinement and training data are needed to reliably handle

complex coding scenarios. Expert evaluations reinforced this point, highlighting that more comprehensive examples and expanded training material would improve performance for advanced use cases.

Regarding RQ2, the system demonstrated a strong ability to answer factual questions accurately. When asked to explain HydroLang and its applications, HydroSuite-AI provided detailed, contextually relevant responses drawn directly from the indexed documentation (Table 3). These results indicate that the assistant effectively leverages NLP and retrieval-augmented generation to provide precise and reliable answers. Nonetheless, the accuracy of responses decreased slightly for complex or nuanced queries, emphasizing the need for ongoing updates to the underlying documentation to ensure the system remains current with evolving library features.

For RQ3, HydroSuite-AI's performance on advanced hydrological queries was mixed. The assistant excelled at providing preliminary insights and code templates for basic tasks, such as describing library functionalities and generating simple code snippets. Yet, for sophisticated analyses for example, implementing a MonteCarlo-ARIMA model with real data using HydroCompute the system provided only generic guidance and not a fully tailored solution (Table 3). This indicates that while the assistant can serve as a starting point for complex workflows, additional training and curated examples are necessary for it to deliver fully comprehensive responses.

Finally, in addressing RQ4, HydroSuite-AI's integration of HydroLang, HydroCompute, and HydroRTC into a unified platform demonstrates promising potential for cross-interoperability beyond the initial libraries. The collaborative peer-to-peer case study (Fig. 7) illustrates the assistant's ability to orchestrate end-to-end workflows, including data retrieval, reprojection, visualization, and peer-to-peer transfer. While the system generally maintained variable consistency and successfully facilitated multi-library operations, evaluations identified occasional challenges when integrating external data sources or coordinating multiple modules simultaneously. These results highlight both the progress made and the areas requiring further enhancement to fully automate complex hydrological workflows.

Towards a generalized tool for hydrological agentic workflows

The results above come from three libraries the authors also develop, serving as the initial step towards a generalized tool for agentic hydrological workflows. It is useful, however, to distinguish which parts of the architecture are HydroSuite-specific and which are not, since this determines what generalizing the tool would actually require.

Three components of the Planner-Worker-Synthesizer architecture are library-agnostic by construction. The retrieval-augmented grounding step indexes API documentation into embeddings, a process that is mechanically identical regardless of which library's documentation is indexed. The worker-agent decomposition assigns sub-tasks by function domain agnostic of library identity. The virtualized, browser-based execution environment validates generated code against a sandboxed runtime independent of the libraries being called. None of these three components depend on properties specific to the HydroSuite libraries.

Two components have not yet been shown to generalize. The benchmarking protocol in Table 5 assumes a curated set of documented tasks with known correct function calls, and building this set requires domain expertise in the target library, which is a superficial bottleneck in extending evaluation to a new package. The interoperability case study in Fig. 7 relied on the main testing libraries already sharing common data interchange conventions, such as the Basic Modeling Interface and common raster or array formats. Cross-library orchestration for packages without such shared conventions remains untested, and the multi-library failure modes will be included as future work.

On this basis, generalization is best treated as a specific and testable hypothesis: the same architecture, applied to a different well-documented scientific software package, should reproduce comparable FRA and FHR figures without changes beyond re-indexing the documentation and rebuilding the benchmark task set.

Limitations and future work

HydroSuite-AI has several concrete limitations. First, its performance depends entirely on the quality and scope of its training data, which is limited to the documentation of the target libraries. Queries involving other libraries, external tools, or new features released after training may produce incomplete or incorrect results. Second, the current embedding configuration (256-dimensional vector embeddings) may not fully capture the complexity of hydrological queries, reducing search precision. Increasing the embedding size could improve relevance but would also increase memory usage and computational cost.

The system also relies on accurate and up-to-date documentation. Any changes in library APIs or function parameters not reflected in the training data can lead to incorrect code generation or misguidance in workflow implementation. Additionally, the assistant lacks fine-grained control over its search and code generation processes, limiting optimization for complex or multi-step workflows.

Future work should focus on addressing these limitations. Expanding and regularly updating the training data

to include the latest library versions and additional hydrology-relevant tools is critical. Benchmark iteration should also include naive, non-expert-phrased prompts to assess robustness across varying levels of user domain expertise. Adjusting embedding dimensions or exploring alternative vector representations could improve retrieval accuracy for complex queries. Integrating support for additional LLM models optimized for code, such as CodeGen, StarCoder, WizardCoder, or CodeT5 (Coignon et al. 2024; Guo et al. 2024; Idrisov and Schlippe 2024), may enhance code generation reliability. Finally, adding mechanisms for verifying generated code against library specifications or automated testing would reduce errors and improve trust in workflow outputs (Jansen et al. 2023; Zapkus and Slotkieni  2024).

Conclusions

This work introduced HydroSuite-AI, an LLM-based assistant that integrates HydroLang, HydroCompute, and HydroRTC through a Planner–Worker–Synthesizer multi-agent architecture. The system combines retrieval-augmented generation grounded in library documentation with a browser-based virtualized execution environment to support factual querying and executable code generation for hydrological workflows.

Evaluation was carried out using a qualitative expert assessment, a cross-library interoperability case study, and an automated benchmark consisting of 60 documented, domain-specific tasks executed across multiple LLM backends. Results show that the system retrieves correct library functions with high reliability under documentation-grounded configurations, with clear accuracy–latency differences across model backends. Higher-accuracy models demonstrated stronger adherence to documented APIs, while faster models supported interactive use with increased error rates.

The interoperability case study confirmed that HydroSuite-AI can generate end-to-end workflows spanning data retrieval, client-side computation, geospatial visualization, and peer-to-peer data transfer while maintaining consistency across library boundaries. The virtualized execution environment enabled validation of generated applications in a self-contained, offline-capable setting. Failure analysis identified recurring limitations in parameter specification and complex workflow synthesis, particularly for high-complexity, multi-step tasks, reflecting constraints in available documentation and training examples.

Despite these limitations, HydroSuite-AI demonstrates that documentation-grounded, multi-agent LLM orchestration can reliably resolve the function-lookup and

boilerplate-generation step of hydrological software development, with reliability that degrades predictably as task complexity increases.

Three of the architecture’s core components including retrieval grounding, worker-agent decomposition, and virtualized execution, do not depend on properties specific to the HydroSuite libraries. This is a necessary but not sufficient condition for generalizing the approach to other scientific software ecosystems. Establishing sufficiency for proper modelling test would require comparable FRA and FHR performance on a library outside hydrology, together with a controlled comparison of assisted and unassisted development time. Furthermore, this work enables the next step in generating full scale hydrological modelling and evaluation through agent driven execution.

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s12145-026-02222-7>.

Acknowledgements This project was funded by the National Oceanic and Atmospheric Administration (NOAA) via a cooperative agreement with the University of Alabama (NA22NWS4320003) awarded to the Cooperative Institute for Research to Operations in Hydrology (CIROH).

Author contributions Vinay Pursnani: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data Curation, Writing - Original Draft, and Visualization. Carlos Erazo: Methodology, Validation, Formal analysis, Writing - Original Draft. Yusuf Sermet: Conceptualization, Methodology, Writing - Review & Editing, Investigation, Validation, and Visualization. Ibrahim Demir: Conceptualization, Methodology, Writing - Review & Editing, Project administration, Supervision, Funding acquisition, and Resources.

Data availability All data generated and analyzed during this study are included in the supplementary materials of this article. The benchmark dataset (S1), generation prompt (S2), graded generation details (S3), generation metrics (S4), multi-model benchmark outputs (S5), and qualitative assessment prompts (S6) are provided as supplementary files.

Software availability Software Name: HydroSuite-AI; Developers: Vinay Pursnani, Carlos Erazo; Contact information: vinay-pursnani@uiowa.edu; Cost: Free; Software required: Web Browser; Program language: Python, JavaScript; Software Availability: <https://hydroinformatics.tulane.edu/lab/hydrosuite-ai-coder/>.

Declarations

Competing interests The authors declare no competing interests.

Generative AI and AI-assisted technologies During the preparation of this work, the authors used ChatGPT to improve the flow of the text, correct any potential grammatical errors, and improve the writing. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Alkour I, Alzyoud M, Alquqa E, Tariq E, Alzboun N, Al-Hawary S, Alshurideh M (2024) Artificial intelligence and financial decisions: Empirical evidence from developing economies. *Int J Data Netw Sci* 8(1):101–108
- Al-Kaswan A, Izadi M (2023), May The (ab) use of open-source code to train large language models. In 2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE): 9–10. IEEE
- Baydaroglu Ö, Yeşilköy S, Sermet Y, Demir I (2023) A comprehensive review of ontologies in the hydrology towards guiding next generation artificial intelligence applications. *J Environ Inf* 42(2):90–75
- Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., ... & Zhang, Y. (2023). Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*. <https://doi.org/10.48550/arXiv.2303.12712>
- Cambaz, D., & Zhang, X. (2024). Use of AI-driven code generation models in teaching and learning programming: A systematic literature review. *Proc ACM Conf Computer-Supported Coop Work Social Comput*. <https://doi.org/10.1145/3626252.3630958>
- Coignon T (2024) A performance study of LLM-generated code on LeetCode. *Proc ACM Conf Comput Commun Secur*. <https://doi.org/10.1145/3661167.3661221>
- Cooper N., Clark, A. T., Lecomte, N., Qiao, H., & Ellison, A. M. (2024) Harnessing large language models for coding, teaching, and inclusion to empower research in ecology and evolution. *OSF Preprints*. <https://doi.org/10.32942/x2ps48>
- Douglas MR (2023) Large language models [Preprint]. *arXiv*. <https://doi.org/10.48550/arxiv.2307.05782>
- Erazo Ramirez C, Sermet Y, Demir I (2024a) HydroCompute: An open-source web-based computational library for hydrology and environmental sciences. *Environ Model Softw* 175:106005. <https://doi.org/10.1016/j.envsoft.2024.106005>
- Erazo Ramirez C, Sermet Y, Demir I (2023) HydroLang markup language: Community-driven web components for hydrological analyses. *J Hydroinformatics* 25(4):1171–1187. <https://doi.org/10.2166/hydro.2023.149>
- Erazo Ramirez C, Sermet Y, Molkenthin F, Demir I (2022) HydroLang: An open-source web-based programming framework for hydrological sciences. *Environ Model Softw* 157:105525. <https://doi.org/10.1016/j.envsoft.2022.105525>
- Erazo Ramirez, C., Sermet, Y., Shahid, M., & Demir, I. (2024b). HydroRTC: A web-based data transfer and communication library for collaborative data processing and sharing in the hydrological domain. *Environmental Modelling & Software*, 178:06068. <https://doi.org/10.1016/j.envsoft.2024.106068>
- Ewing G, Erazo Ramirez C, Vaidya A, Demir I (2024) Client-side web-based model coupling using basic model interface for hydrology and water resources. *J Hydroinformatics* 26(2):494–502. <https://doi.org/10.2166/hydro.2024.212>
- Funk C, Peterson P, Landsfeld M, Pedreros D, Verdin J, Shukla S, Michaelsen J (2015) The climate hazards infrared precipitation with stations—a new environmental record for monitoring extremes. *Sci data* 2(1):1–21
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*. <https://doi.org/10.48550/arXiv.2312.10997>
- Gartlehner G, Kahwati L, Hilscher R, Others (2024) Data extraction for evidence synthesis using a large language model: A proof-of-concept study. *Res Synthesis Methods* 15(4):576–589. <https://doi.org/10.1002/jrsm.1710>
- Guo D, Zhu Q, Yang D, Xie Z, Dong K, Zhang W, Chen G, Bi X, Wu Y, Li YK et al (2024) DeepSeek-Coder: When the large language model meets programming—The rise of code intelligence [Preprint]. *arXiv*. <https://arxiv.org/abs/2401.14196>
- Herath M, Jayatilaka T, Azamathulla HM, Mandala V, Rathnayake N, Rathnayake U (2023) Sensitivity analysis of parameters affecting wetland water levels: A study of flood detention basin, Colombo, Sri Lanka. *Sensors* 23(7):3680. <https://doi.org/10.3390/s23073680>
- Hicks C (2024) The new developer: AI skill threat, identity change & developer thriving in the transition to AI-assisted software development. *OSF Preprints*. <https://doi.org/10.31234/osf.io/2gej5>
- Huespe IA, Echeverri J, Khalid A, Carboni Bisso I, Musso CG, Surani S, Kashyap R (2023) Clinical research with large language models generated writing—Clinical research with AI-assisted writing (CRAW) study. *Crit Care Explorations* 5(10):e0975. <https://doi.org/10.1097/ccx.0000000000000975>
- Idrisov B, Schlippe T (2024) Program code generation with generative AIs. *Algorithms* 17(2):62
- Improta C (2023) Poisoning programs by un-repairing code: Security concerns of AI-generated code. 2023 IEEE 8th Int Conf Softw Secur Reliab Workshops (ISSREW). <https://doi.org/10.1109/issrew60843.2023.00060>
- Jansen J (2023) Leveraging large language models for data analysis automation. *bioRxiv*. <https://doi.org/10.1101/2023.12.11.571140>
- Kadiyala L, Mermer O, Samuel DJ, Sermet Y, Demir I (2024) A comprehensive evaluation of multimodal large language models in hydrological applications. *EarthArXiv*. <https://doi.org/10.31223/x5tq37>
- Krajewski WF, Ghimire GR, Demir I, Mantilla R (2021) Real-time streamflow forecasting: AI vs. hydrologic insights. *J Hydrology* X 13:100110
- Le T (2024) A study of vulnerability repair in JavaScript programs with large language models. *Proc ACM Program Lang* 24:666–669. <https://doi.org/10.1145/3589335.3651463>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://doi.org/10.48550/arXiv.2005.11401>
- Liu F, Liu Y, Shi L, Huang H, Wang R, Yang Z, Zhang L (2024) Exploring and evaluating hallucinations in LLM-powered code generation [Preprint]. *arXiv*. <https://arxiv.org/abs/2404.00971>
- Liu Z (2024) No need to lift a finger anymore. Evaluating code generation quality by ChatGPT. *IEEE Trans Software Eng* 50(6):1548–1584. <https://doi.org/10.1109/tse.2024.3392499>
- Li Z, Duque FQ, Grout T, Bates B, Demir I (2023) Comparative analysis of performance and mechanisms of flood inundation map generation using Height Above Nearest Drainage, vol 159. *Environmental Modelling & Software*, p 105565
- MacNeil S, Kim J, Leinonen J, Denny P, Bernstein S, Becker B, Kumar V (2023) The implications of large language models for

- CS teachers and students. *Proc 54th ACM Tech Symp Comput Sci Educ*. <https://doi.org/10.1145/3545947.3573358>
- Mosavi A, Öztürk P, Chau KW (2018) Flood prediction using machine learning models: Literature review. *Water* 10(11):1536. <https://doi.org/10.3390/w10111536>
- Pan Z (2024) Exploring the key factors influencing college students' willingness to use AI coding assistant tools: An expanded technology acceptance model. *Systems* 12(5):176. <https://doi.org/10.3390/systems12050176>
- Peng B, Galley M, He P, Cheng H, Xie Y, Yu H, Gao J (2023) Check your facts and try again: Improving large language models with external knowledge and automated feedback [Preprint]. *arXiv*. <https://doi.org/10.48550/arxiv.2302.12813>
- Perera D, Agnihotri J, Seidou O, Djalante R (2020) Identifying societal challenges in flood early warning systems. *Int J Disaster Risk Reduct* 51:101794. <https://doi.org/10.1016/j.ijdrr.2020.101794>
- Pursnani V, Sermet Y, Demir I, Pursnani V, Sermet Y, Kurt M, Demir I (2025) A conversational intelligent assistant for enhanced operational support in floodplain management with multimodal data. *Int J Disaster Risk Reduction Computers Education: Artif Intell* 122:105422
- Pursnani V, Sermet Y, Kurt M, Demir I (2023) Performance of ChatGPT on the US fundamentals of engineering exam: Comprehensive assessment of proficiency and potential implications for professional environmental engineering practice. *Computers Education: Artif Intell* 5:100183. <https://doi.org/10.1016/j.caeai.2023.100183>
- Ren Y, Zhang T, Dong X, Li W, Wang Z, He J, Zhang H, Jiao L (2024) WaterGPT: Training a large language model to become a hydrology expert. *Water* 16(21):3075
- Sajja R, Sermet Y, Cikmaz M, Cwierny D, Demir I (2024) Artificial intelligence-enabled intelligent assistant for personalized and adaptive learning in higher education. *Information* 15(10):596
- Samuel DJ, Sermet MY, Mount J, Vald G, Cwierny D, Demir I (2024b) Application of large language models in developing conversational agents for water quality education, communication and operations. *EarthArXiv*. <https://doi.org/10.31223/x5xt4k>
- Samuel DJ, Sermet Y, Cwierny D, Demir I (2024a) Integrating vision-based AI and large language models for real-time water pollution surveillance. *Water Environ Res* 96(8):e11092
- Sermet Y, Demir I (2021) A semantic web framework for automated smart assistants: A case study for public health. *Big Data Cogn Comput* 5(4):57. <https://doi.org/10.3390/bdcc5040057>
- Shahid M, Sermet Y, Mount J, Demir I (2023) Towards progressive geospatial information processing on web systems: A case study for watershed analysis in Iowa. *Earth Sci Inf* 16(2):1597–1610
- Sit M, Demiray B, Demir I (2021b) Short-term hourly streamflow prediction with graph convolutional GRU networks [Preprint]. *arXiv*. <https://arxiv.org/abs/275.07039>
- Sit M, Langel RJ, Thompson D, Cwierny DM, Demir I (2021a) Web-based data analytics framework for well forecasting and groundwater quality. *Sci Total Environ* 761:144121
- Song H (2023) Evaluating the performance of different large language models on health consultation and patient education in urolithiasis. *Res Square*. <https://doi.org/10.21203/rs.3.rs-3293294/v1>
- Tai-Seale M, Baxter SL, Vaida F, Others (2024) AI-generated draft replies integrated into health records and physicians' electronic communication. *JAMA Netw Open* 7(4):e246565. <https://doi.org/10.1001/jamanetworkopen.2024.6565>
- Tegegne G, Park DK, Kim Y (2017) Comparison of hydrological models for the assessment of water resources in a data-scarce region, the upper Blue Nile River basin. *J Hydrol Reg Stud* 14:49–66. <https://doi.org/10.1016/j.ejrh.2017.10.002>
- USGS (2023) What is 3DEP? Accessible at: <https://www.usgs.gov/3d-elevation-program>
- Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. *Advances in Neural Information Processing Systems*, vol 30. Curran Associates
- WaterSoftHack (2024), July 22 WaterSoftHack landing website. <https://watersofthack.github.io/>
- Wong M, Guo S, Hang C, Ho S, Tan C (2023) Natural language generation and understanding of big code for AI-assisted programming: A review. *Entropy* 25(6):888. <https://doi.org/10.3390/e25060888>
- Xu F, Vasilescu B, Neubig G (2022) In-IDE code generation from natural language: Promise and challenges. *ACM Trans Softw Eng Methodol* 31(2):1–47. <https://doi.org/10.1145/3487569>
- Yeo S (2024) Framework for evaluating code generation ability of large language models. *ETRI J* 46(1):106–117. <https://doi.org/10.4218/etrij.2023-0357>
- Zapkus D (2024) Unit test generation using large language models: A systematic literature review. *Vilnius Univ Open Ser* 136–144. <https://doi.org/10.15388/lmitt.2024.20>
- Zhang H, Xu J-J (2023) When geoscience meets foundation models: Towards general geoscience artificial intelligence system [Preprint]. *arXiv*. <https://arxiv.org/abs/2309.06799>
- Zhong L (2024) Can LLM replace Stack Overflow? A study on robustness and reliability of large language model code generation. *Proc AAAI Conf Artif Intell* 38(19):21841–21849. <https://doi.org/10.1609/aaai.v38i19.30185>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.